EX25 · Asignación de fuerzas con álgebra lineal

Ejercicio práctico de ingeniería · estimación 60–90 min · conocimientos previos: funciones, NumPy, JSON y pytest.

LANGUAGE / ID	UNIT	HANDOUT
ES · EX25	algebra / vectors and linear systems	Ejercicio / Misión

1 · Objetivo de la misión

Una plataforma dispone de **dos actuadores ideales** que generan fuerzas en direcciones fijas dentro de un plano. Tu software debe calcular cuántos newtons aporta cada actuador para producir una fuerza objetivo, verificar la solución y estudiar cómo cambia si modificamos el objetivo.

> **El reto:** convertir un modelo físico en un sistema lineal, implementar funciones reutilizables, comprobar unidades y magnitudes y verificar el código con 10 tests automáticos.

2 · Modelo de ingeniería

Representamos las direcciones unitarias de los dos actuadores como columnas de una matriz A. El vector u contiene las magnitudes de las fuerzas y b es la fuerza resultante deseada.

$$Au = b$$

```
A = [[ 0.8,  0.6],
     [ 0.6, -0.8]]

b       = [14.00, 2.00]      # target Fx,Fy [N]
delta_b = [ 0.00, 0.01]      # variation [N]

# Solve for u = [u1, u2], in newtons.
```

La primera columna de A es la dirección del actuador 1: (0,8; 0,6). La segunda es la del actuador 2: (0,6; −0,8). Cada dirección tiene módulo uno. Suponemos que las fuerzas actúan en un mismo punto, sin pares ni límites de saturación.

3 · Entregable y ubicación exacta

```
Python_Engineering/
└── 02_numerical_methods/
    └── algebra/
        └── class_17_vectors_and_linear_systems/
            ├── main.py
            ├── inputs/actuators_case.json
            ├── src/algebra.py       # complete the TODOs
            ├── tests/test_algebra.py
            ├── docs/ES/ and docs/EN/
            └── outputs/
```

El proyecto suministrado incluye los archivos base y los tests. **No hay solución implementada:** los NotImplementedError son intencionados.

4 · Funciones que debes implementar

Función	Entrada	Salida / comprobación
resultant_force(A, u)	Matriz 2×2; magnitudes (2,)	Vector de fuerzas A @ u, forma (2,)
solve_forces(A, b)	Matriz 2×2; objetivo (2,)	Magnitudes u; validar dimensiones y finitud
residual_norm(A, u, b)	A, solución u, objetivo b	Número real: norma del residual en N
relative_residual(A, u, b)	A, u y b	Número sin unidades; rechazar ||b|| = 0
sensitivity_analysis(A, b, delta_b)	A, objetivo y perturbación	Diccionario con las 7 claves del docstring

Paso 1 · Operar con vectores y matrices

- Implementa **resultant_force** mediante el operador @, no el producto elemento a elemento (*).
- Revisa **shape**, tipos y significado físico de cada componente.

Paso 2 · Resolver el sistema

- Implementa **solve_forces** con **np.linalg.solve**. No uses la matriz inversa explícitamente.
- Usa **ValueError** para tamaños erróneos o NaN/Inf y deja pasar **np.linalg.LinAlgError** si la matriz es singular.

Paso 3 · Cuantificar el error de reconstrucción

- Calcula el residual absoluto: $||A @ u - b||_2$, en newtons.
- Normaliza ese residual por $||b||_2$ para el residual relativo, que es adimensional.

Paso 4 · Estudiar sensibilidad

- Resuelve dos veces: primero A @ u = b y luego A @ u_perturbed = b + delta_b.
- Obtén delta_u, su norma, los cambios relativos de entrada y salida y **np.linalg.cond(A, 2)**.

5 · Resultados esperados para validar

Comprobación	Resultado de referencia	Tolerancia / unidad
u original	[12.4, 6.8]	10^{-10} N
A @ u	[14.0, 2.0]	10^{-10} N
Residual absoluto y relativo	Próximos a 0	Ambos < 10^{-10}
u perturbado	[12.406, 6.792]	10^{-10} N
delta_u	[0.006, −0.008]	10^{-10} N
$\|\|delta_u\|\|_2$	0.010000	N
Condición $\kappa_2(A)$	1.000000	Adimensional

El cambio relativo de b y el de u son ambos aproximadamente 0,0007071, es decir, **0,07071 %**. Interpreta por qué coinciden en este caso.

6 · Pruebas y ejecución

```
# VS Code terminal, inside class_17_vectors_and_linear_systems
python -m pip install -r requirements.txt
python -m pytest -q
python main.py
```

Los 10 tests cubren cálculo directo, solución correcta, residuales, objetivo nulo, dimensiones inválidas, datos no finitos, matriz singular y sensibilidad. **Al principio fallarán** porque las funciones están por completar.

7 · Preguntas de ingeniería para tu entrega

- ¿Qué representa cada columna de A? ¿En qué unidad se expresan u y b?
- ¿Por qué una solución con residual pequeño no garantiza siempre una solución fiable?
- ¿Qué significa $\kappa_2(A) = 1$ en esta geometría?
- ¿Qué cambiaría físicamente si un actuador solo pudiera generar fuerzas positivas y tuviera una fuerza máxima?

8 · Extensión opcional

Construye una segunda matriz con direcciones casi paralelas (por ejemplo, filas [1, 1] y [1, 1,000001]). Resuelve un caso antes y después de una pequeña perturbación y compara el número de condición. No confundas una solución matemática con una distribución físicamente realizable.

> **Criterio de finalización:** 10 tests superados, ejecución de main.py, comparación con valores esperados y explicación breve del significado del residual y del número de condición.